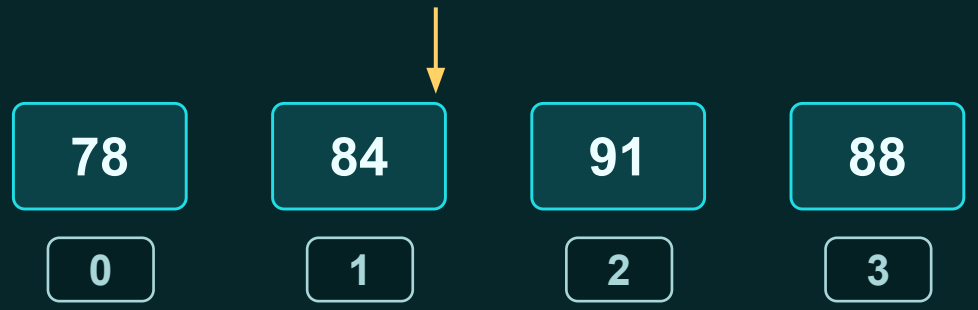
BLOCK 17

LISTS & ORDERED COLLECTIONS

Block 17 • Grouping values before automating them

A list lets one variable refer to many related values in a known order.

```
scores = [78, 84, 91, 88]
```



Today: one name, many values, fixed order, readable access.

BLOCK 17

Why collections are needed

Four scores are manageable. Four thousand are not.

```
score_1 = 78
score_2 = 84
score_3 = 91
score_4 = 88
```

```
scores = [78, 84, 91, 88]
```

Collection idea

- Treat related values as one group
- Keep order
- Access items by position
- Later: automate with loops

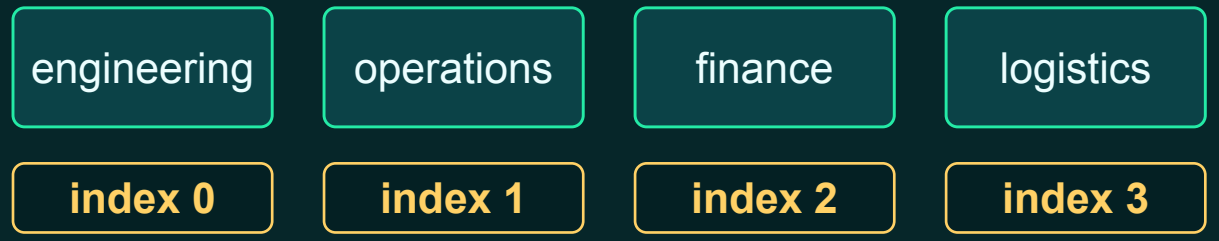
Lists solve the “too many separate variable names” problem.

BLOCK 17

A list is an ordered collection

Order matters because each item has a position.

```
departments = [
    "engineering",
    "operations",
    "finance",
    "logistics"
]
```



Python starts counting positions at zero.

That feels weird at first. It becomes normal fast.

The order is part of the data.

BLOCK 17

Creating lists

Square brackets define the collection. Commas separate items.

```
scores = [78, 84, 91, 88]

record_ids = ["EQ-101", "EQ-102", "EQ-103"]

empty_errors = []
```

Usually easier

A list of similar things:
- all scores
- all record IDs
- all validation messages

Possible, but often messy

Python allows mixed types:
["EQ-101", 72.5, True]

But consistent data is easier to process.

Lists can be empty at first and filled later.

BLOCK 17

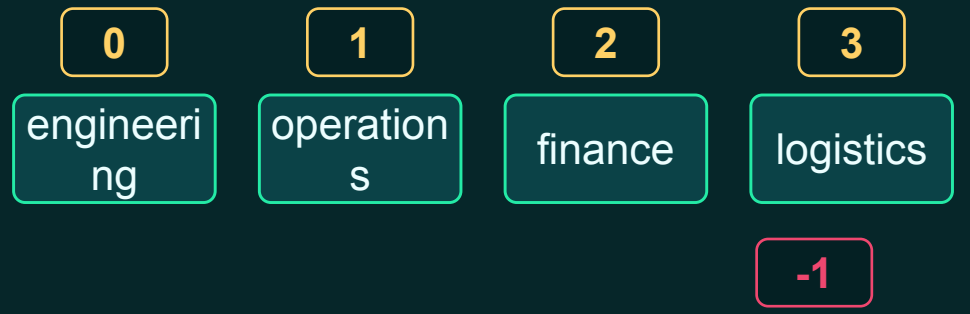
Accessing items by index

Use brackets with a position number.

```
departments = ["engineering", "operations",
"finance", "logistics"]

print(departments[0])
print(departments[2])
print(departments[-1])
```

Negative indexes count backward from the end.



departments[0] returns the first item. departments[-1] returns the last item.

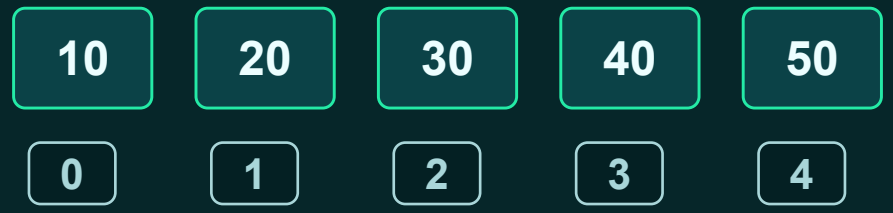
BLOCK 17

Slicing: selecting a range

Same start:end pattern as strings. The end is not included.

```
values = [10, 20, 30, 40, 50]

first_three = values[0:3]
middle_values = values[1:4]
```



values[0:3] gives [10, 20, 30]

Start at 0. Stop before 3.

Slice thinking: include the start, exclude the stop.

BLOCK 17

List length and position errors

len() tells how many items are in the collection.

```
record_ids = ["EQ-101", "EQ-102", "EQ-103"]

print(len(record_ids))

print(record_ids[3])  # problem
```

Length is 3

Valid indexes are:
0, 1, and 2

Index 3 is past the end.

Common beginner rule

Last positive index = len(list) - 1

Three items does not mean index 3 exists.

BLOCK 17

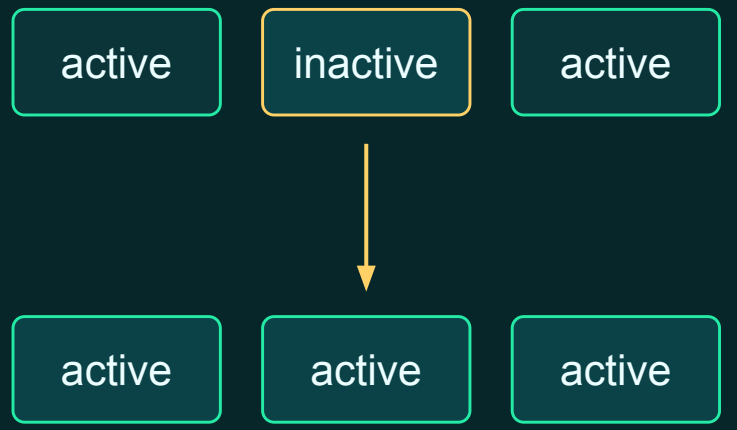
Lists are mutable

Mutable means the list can be changed after it is created.

```
statuses = ["active", "inactive",
"active"]

statuses[1] = "active"

print(statuses)
```



Unlike strings, a list item can be replaced at a position.

Changing a list item changes the existing list.

BLOCK 17

Adding items

append adds to the end. insert adds at a chosen position.

```
record_ids = ["EQ-101", "EQ-102"]

record_ids.append("EQ-103")

record_ids.insert(1, "EQ-101A")
```

.append(value)

Add one item to the end of the list.

.insert(index, value)

Place one item before a position.

These methods change the list.

Adding items is the first step toward collecting results.

BLOCK 17

Removing items

Remove by value or remove by position.

```
record_ids.remove("EQ-101A")

removed_record = record_ids.pop()

print(removed_record)
print(record_ids)
```

.remove(value)

Find this value and remove it.
Good when you know the value.

.pop()

Remove and return an item.
With no index, it removes the last item.

pop is useful when the removed value still matters.

BLOCK 17

Membership testing with in

Lists make allowed-value checks much cleaner.

```
supported_extensions = ["csv", "json", "txt"]

is_supported = "csv" in supported_extensions
```

Why this matters

Allowed values are data too.

A list lets the rule read like English.

```
is_supported = (
    extension == "csv"
    or extension == "json"
    or extension == "txt"
)
```

This directly improves the validation logic from Day 2.

BLOCK 17

Empty lists collect results

Start with nothing. Add messages as problems are found.

```
validation_errors = []

if record_id.strip() == "":
    validation_errors.append("Record ID is
missing")

if file_size <= 0:
    validation_errors.append("File size must be
greater than zero")
```

Pattern

1. Create empty list
2. Check each rule
3. Append every problem
4. Report the list

This supports “collect many problems” instead of stopping at the first failure.

BLOCK 17

Mixed types are possible, but consistency helps

Python allows flexibility. Data work rewards predictability.

```
mixed_record = ["EQ-101", 72.5, True]

temperatures = [72.5, 75.1, 81.2, 68.9]

record_ids = ["EQ-101", "EQ-102", "EQ-103"]
```

Consistent lists are easier to:

- validate
- calculate with
- sort
- display
- process later with loops

Rule of thumb: group values that represent the same kind of thing.

Python will let you mix values. That does not mean you should.

BLOCK 17

Guided exercise 1: Measurements

Use a list of temperatures and practice basic operations.

```
temperatures = [72.5, 75.1, 81.2, 68.9, 77.0]
```

Tasks

- Print the first value
- Print the last value
- Print the number of values
- Change one temperature
- Add another temperature
- Remove one temperature

Instructor cue

Have students predict the list after each operation before running the code.

Do one operation at a time. Print after each change.

BLOCK 17

Guided exercise 2: Supported file types

Normalize first, then test membership.

```
supported_types = ["csv", "json", "parquet"]

extension = "PARQUET"
normalized_extension = extension.strip().lower()

is_supported = normalized_extension in supported_types
```

Test these values

- "csv"
- "txt"
- "PARQUET" before normalization
- "PARQUET" after normalization

Key idea: raw text often needs cleanup before comparison.

This ties lists back to string normalization and validation.

BLOCK 17

Guided exercise 3: Validation errors

Build a list of every problem found.

```
validation_errors = []

record_id = ""
file_size = 0
error_count = -2
```

```
if record_id.strip() == "":
    validation_errors.append("Record ID is
missing")

if file_size <= 0:
    validation_errors.append("File size must be
greater than zero")

if error_count < 0:
    validation_errors.append("Error count cannot
be negative")
```

Print

- the error list
- the number of errors
- the first error

The error list becomes a simple report.

BLOCK 17

Guided exercise 4: Slicing records

Practice taking the front, middle, and end of a list.

```
record_ids = [
    "EQ-1001",
    "EQ-1002",
    "EQ-1003",
    "EQ-1004",
    "EQ-1005",
]
```

Extract

- First two records
- Last two records
- Middle three records

```
first_two = record_ids[0:2]
last_two = record_ids[-2:]
middle_three = record_ids[1:4]
```

Slicing lets us inspect a subset without changing the original list.

BLOCK 17

Applied challenge: data-quality tracker

Manual inspection today. Automation comes next.

```
record_ids = [
    "EQ-1001",
    "",
    "EQ-1003",
    "BAD-1004",
]

invalid_records = []
```

Goal

Without loops yet, inspect each position manually and add invalid values to invalid_records.

Invalid means

- blank value
- does not begin with EQ-

This sets up why loops are useful.

BLOCK 17

Manual inspection pattern

Check one position at a time. Append bad values.

```
record = record_ids[1]

if record.strip() == "" or not
record.startswith("EQ-"):
    invalid_records.append(record)

record = record_ids[3]

if record.strip() == "" or not
record.startswith("EQ-"):
    invalid_records.append(record)
```

Important limitation

This works for four records.

It would be painful for four thousand.

Tomorrow idea preview

A loop repeats the same logic for every item.

Manual repetition is the motivation for loops.

BLOCK 17

Challenge output

Report the collection and explain the result.

```
print(f"Records examined: {len(record_ids)}")
print(f"Invalid records:
{len(invalid_records)}")
print(f"Invalid values: {invalid_records}")
```

Explain in plain language

The list stored every invalid value found during the checks.

```
Records examined: 4
Invalid records: 2
Invalid values: ["", "BAD-1004"]
```

Lists help the program remember more than one result.

BLOCK 17

Common mistakes

Most list bugs are about position, mutation, or assumptions.

Index confusion

- First item is index 0
- Last positive index is len(list) - 1
- Index equal to len(list) is too far

Method confusion

- append adds to the end
- insert needs a position
- remove looks for a value
- pop removes by position

Data confusion

- "CSV" is not "csv"
- mixed types complicate processing
- empty strings still count as values

Debugging move: print the list after every change.

BLOCK 17

Mini knowledge check

Students should answer without running code first.

```
values = [10, 20, 30, 40]

print(values[0])
print(values[-1])
print(values[1:3])

values.append(50)
print(len(values))
```

Predict

1. First output
2. Last output before append
3. Slice result
4. Length after append

Then run it and compare.

Prediction builds execution-tracing skill.

BLOCK 17

Discussion: when should values be grouped?

Lists are a design choice, not just syntax.

Good list candidates

- temperatures
- record IDs
- filenames
- validation errors
- supported extensions

Maybe not a list

A single record with different fields:
- name
- status
- hours
- approval

Later we will learn better structures for this.

Plain-language question

“Do these values represent the same kind of thing?”

Data modeling starts with simple grouping decisions.

BLOCK 17

Wrap-up: collections unlock automation

Today we grouped values. Next, we repeat work across them.

Students can now explain

- why lists exist
- how list positions work
- how to add, remove, and update values
- how lists support validation

```
validation_errors = []

# Today: add messages manually
# Next: repeat checks automatically
```

One list + one repeated rule = the beginning of real data processing.